

Proof of Concept (PoC)

(made under Digisuraksha Parhari Foundation)

This PoC focuses on a CLI-based Python tool named **NetSentinel**, developed to detect suspicious network activities (pings, scans, and exploit attempts) in real time or from packet captures (PCAPs). It provides a lightweight intrusion detection capability useful in SOC environments, red-team exercises, and digital forensics investigations.

Tool: NetSentinel IDS

1. History:

The concept of Intrusion Detection Systems (IDS) gained momentum in the late 1980s with early signature-based approaches. Over time, tools like Snort and Suricata became industry standards. However, such systems are often heavy and resource-intensive.

NetSentinel was developed in 2025 under the Digisuraksha Parhari Foundation as a Python-based CLI IDS for interns to understand the fundamentals of traffic analysis and detection.

2. Description:

NetSentinel is a command-line utility that inspects network packets (live or offline PCAPs) and raises alerts for common attack indicators such as ICMP floods, TCP SYN scans, NULL/FIN scans, and repeated suspicious connection attempts.

It provides forensic-grade insights into packet behavior, helping analysts catch reconnaissance or low-level exploit attempts early.

3. What Is This Tool About?

NetSentinel is a **lightweight IDS** that focuses on detecting early signs of scanning, probing, and flooding attacks. Unlike full-fledged enterprise IDS/IPS, it is built to be transparent, minimal, and educational while still producing useful alerts.

4. Key Characteristics / Features:

- Reads packets live from an interface or offline PCAP
- Detects ICMP pings and floods
- Identifies TCP SYN connection attempts and half-open sessions
- Recognizes common scan signatures (SYN, NULL, FIN scans)
- Detects repeated high-rate attempts to multiple ports

- Color-coded terminal alerts (red = critical, yellow = suspicious)
 - Simple CLI workflow (one-liners to run)
 - Dependency-light (scapy + colorama)
-

5. Types / Modules Available:

- Live capture mode (--iface eth0)
 - PCAP analysis mode (--file traffic.pcap)
 - Optional expansion to:
 - JSON/CSV reporting
 - Integration with ELK/SIEM
 - Real-time dashboard (Tkinter/Flask)
-

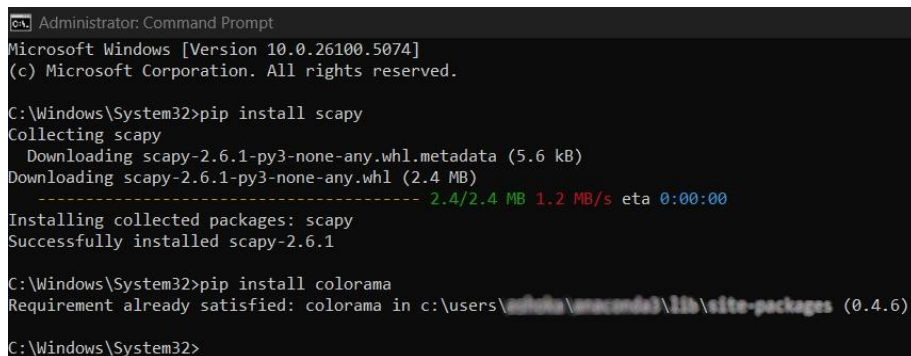
6. How Will This Tool Help?

- Detect reconnaissance (ping sweeps, port scans)
 - Identify basic exploit attempts before escalation
 - Provide alerts during forensic review of PCAPs
 - Help interns understand packet-level behaviors
 - Act as a training-friendly, resource-minimal IDS
-

7. Proof of Concept (PoC) Images:

(*Some details are blurred for safety purposes)

- **Screenshot 1:** Dependencies



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26100.5074]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>pip install scapy
Collecting scapy
  Downloading scapy-2.6.1-py3-none-any.whl.metadata (5.6 kB)
  Downloading scapy-2.6.1-py3-none-any.whl (2.4 MB)
----- 2.4/2.4 MB 1.2 MB/s eta 0:00:00
Installing collected packages: scapy
Successfully installed scapy-2.6.1

C:\Windows\System32>pip install colorama
Requirement already satisfied: colorama in c:\users\mike\appdata\local\site-packages (0.4.6)

C:\Windows\System32>
```

- **Screenshot 2:** Fetching the ip address of “Wireless LAN Adapter Wi-fi”

```

Administrator: Command Prompt
C:\Windows\System32>ipconfig

Windows IP Configuration

Ethernet adapter Ethernet 2:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Unknown adapter Local Area Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet 3:

    Connection-specific DNS Suffix  . :
    IPv4 Address. . . . . : 192.168.255.8
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Wireless LAN adapter Local Area Connection* 8:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::154d:c282:9087:8a2b%1
    IPv4 Address. . . . . : 192.168.255.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::154d:c282:9087:8a2b%8
    IPv4 Address. . . . . : 192.168.255.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Wireless LAN adapter Wi-Fi:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Ethernet adapter VMware Network Adapter VMnet1:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::154d:c282:9087:8a2b%1
    IPv4 Address. . . . . : 192.168.255.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

Ethernet adapter VMware Network Adapter VMnet8:

    Connection-specific DNS Suffix  . :
    Link-local IPv6 Address . . . . . : fe80::154d:c282:9087:8a2b%8
    IPv4 Address. . . . . : 192.168.255.1
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . :

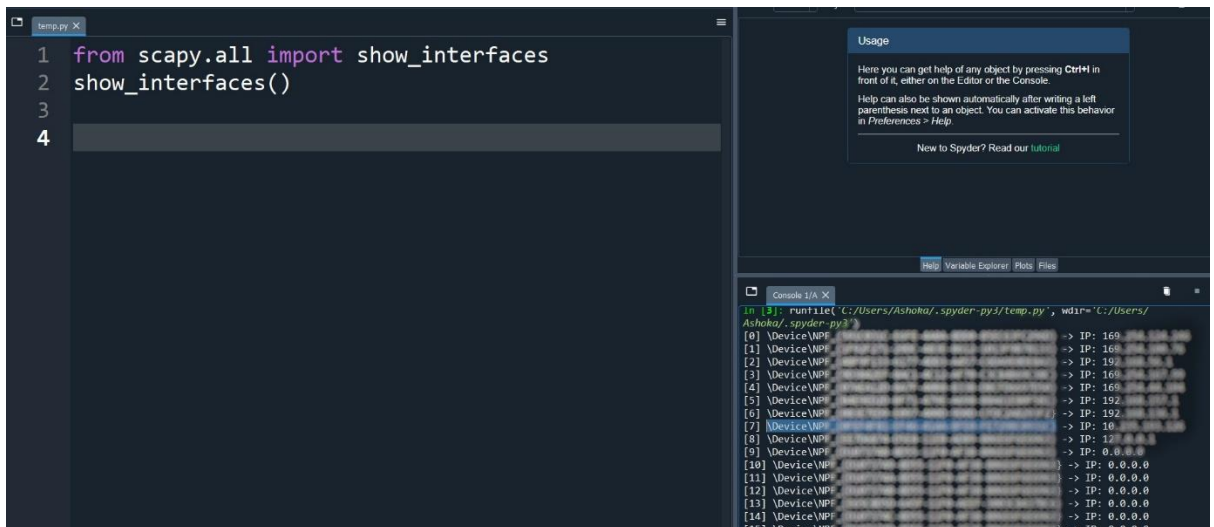
Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . :
    IPv4 Address. . . . . : 192.168.255.8
    Subnet Mask . . . . . : 255.255.255.0
    Default Gateway . . . . . : 192.168.255.250

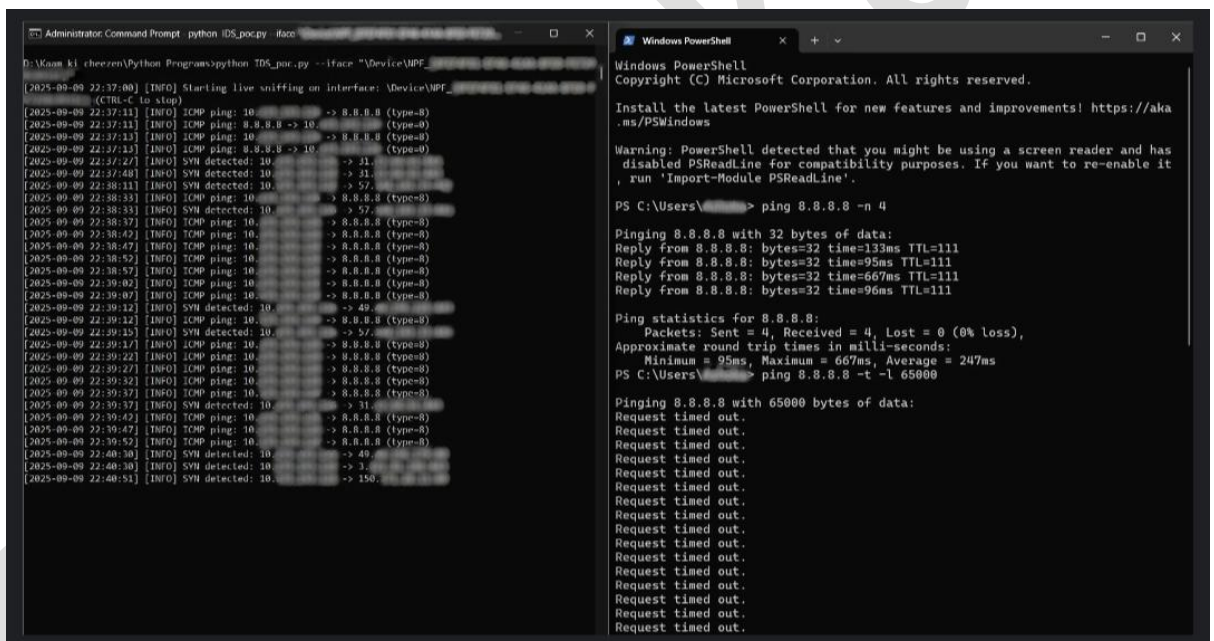
Ethernet adapter Bluetooth Network Connection:

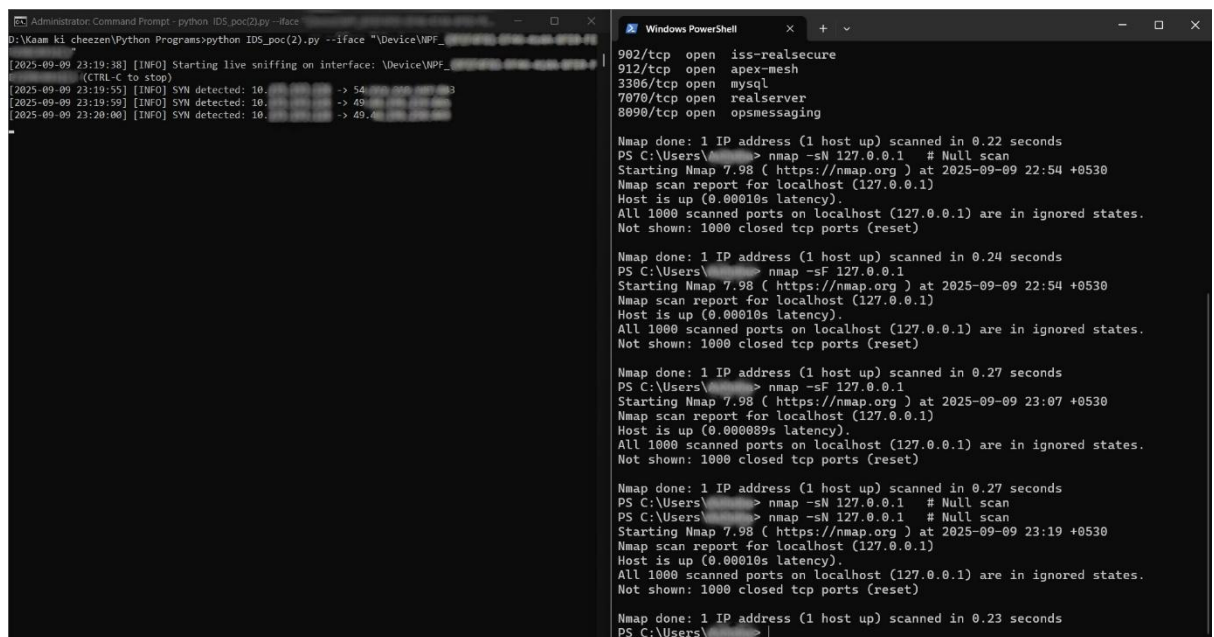
    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :
  
```

- **Screenshot 3:** Fetching the required Windows Npcap or WinPcap device identifier (the blue highlighted one that started from “\Device\NPF....”) by match the ip address on the last column with the ip address fetched earlier.



- Screenshot 4: Ping detections





```
Administrator: Command Prompt - python IDS_poc(2).py --iface
D:\Kaan ki cheezen\Python Programs>python IDS_poc(2).py --iface "\Device\NPF_{...}
[2025-09-09 23:19:38] [INFO] Starting live sniffing on interface: \Device\NPF_{...}
(CTRL-C to stop)
[2025-09-09 23:19:55] [INFO] SYN detected: 10.0.0.1 -> 54.0.0.1
[2025-09-09 23:19:59] [INFO] SYN detected: 10.0.0.1 -> 49.0.0.1
[2025-09-09 23:20:00] [INFO] SYN detected: 10.0.0.1 -> 49.0.0.1

Windows PowerShell
902/tcp open  iss-realsure
912/tcp open  apex-mesh
3306/tcp open  mysql
7070/tcp open  realserver
8090/tcp open  opsmessaging

Nmap done: 1 IP address (1 host up) scanned in 0.22 seconds
PS C:\Users\...> nmap -sN 127.0.0.1 # Null scan
Starting Nmap 7.98 ( https://nmap.org ) at 2025-09-09 22:54 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00010s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
PS C:\Users\...> nmap -sF 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2025-09-09 22:54 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00010s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
PS C:\Users\...> nmap -sF 127.0.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2025-09-09 23:07 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00089s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.27 seconds
PS C:\Users\...> nmap -sN 127.0.0.1 # Null scan
PS C:\Users\...> nmap -sN 127.0.0.1 # Null scan
Starting Nmap 7.98 ( https://nmap.org ) at 2025-09-09 23:19 +0530
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00010s latency).
All 1000 scanned ports on localhost (127.0.0.1) are in ignored states.
Not shown: 1000 closed tcp ports (reset)

Nmap done: 1 IP address (1 host up) scanned in 0.23 seconds
PS C:\Users\...
```

8. ❌ How to Set Up & Run NetSentinel on a System

- Python 3.8+
- Modules: scapy, colorama

a. Install dependencies:

pip install scapy colorama

b. Run in live capture mode:

python netsentinel.py --iface eth0

c. Run in PCAP analysis mode:

python netsentinel.py --file sample.pcap

d. Optional: Create standalone executable:

pip install pyinstaller

pyinstaller --onefile netsentinel.py

9. Usage Scenarios & Examples

a. Detecting Ping Sweeps

python netsentinel.py --file ping_sweep.pcap

b. Identifying SYN Scan in Firewall Logs

python netsentinel.py --iface wlan0

- c. Forensic PCAP Review After Incident
 - d. SOC Daily Traffic Monitoring for Recon Attempts
-

10. 15-Liner Summary:

1. Detects ICMP pings and floods
 2. Identifies TCP SYN attempts
 3. Flags NULL/FIN/SYN scans
 4. Live capture support
 5. PCAP analysis support
 6. CLI based — simple and fast
 7. Color-coded terminal alerts
 8. Low dependency (scapy + colorama)
 9. Alerts on repeated suspicious patterns
 10. Detects early reconnaissance
 11. Supports offline forensic review
 12. Expandable to SIEM/GUI
 13. Python-based & lightweight
 14. Good learning tool for interns
 15. Runs cross-platform
-

11. Time to Use / Best Case Scenarios:

- While analyzing suspicious PCAPs
 - During red-team vs blue-team drills
 - In SOC monitoring for scan activity
 - To quickly triage traffic logs
 - As part of an incident response toolkit
-

12. When to Use During Investigation:

- At the first stage of network forensic analysis
- During detection of brute-force or scan attempts

- While validating IOCs in network captures
 - In real-time detection of suspicious traffic
-

13. Best Person to Use & Required Skills:

Best User:

- SOC Analyst
- Threat Hunter
- DFIR Practitioner
- Security Intern (training)

Skills Needed:

- Basic command-line usage
 - Understanding of TCP/IP basics
 - Awareness of scanning and flooding attack patterns
-

14. Flaws / Suggestions to Improve:

- No DPI (Deep Packet Inspection)
 - Doesn't integrate with existing SIEM tools (yet)
 - Lacks anomaly-based detection (signature-only PoC)
 - No correlation engine for multi-stage attacks
 - Doesn't block traffic (detection-only)
-

15. Good About the Tool:

- Lightweight and educational
- Easy to use and extend
- Great PoC for intern projects
- Provides real-time feedback
- Works cross-platform with Python
- Excellent for triage and quick detection

Real-World Scenario

Consider a corporate network where employees are working remotely through VPN. An attacker launches an **Nmap SYN scan** from a compromised host inside the network, probing hundreds of ports on critical servers. Simultaneously, the attacker performs an **ICMP ping sweep** to map active machines.

Without an IDS, these activities may go unnoticed until a full compromise occurs. **NetSentinel**, when deployed on the VPN gateway or as a forensic tool on captured traffic, will quickly flag:

- High-rate SYNs across multiple ports (possible reconnaissance).
- Large volumes of ICMP requests (possible ping sweep or flood).
- Half-open TCP connections left hanging (potential stealth scan).

This allows the SOC team to **detect the intruder in the reconnaissance stage**, well before data theft or exploitation, reducing risk and response time.

- Made by Sarthaka
Subhankara Singh

Intern id – 438

Digisuraksha Parhari
Foundation